Innovation and Implementation: Multi-Paradigm Medical Insurance Fraud Intelligence

Project: Health Insurance Fraud Intelligence Platform (`Medical Insurance Claim Fraud Detection`)

Institution: Department of Computer Science and Engineering, IIIT Dharwad

Team: B. Varshith (`23BDS011`), J. Ganesh Kumar Reddy (`23BDS024`), M. Jagadeeswar (`23BDS033`) — Supervised by **Prof. Ramesh Athe**

1. Key Architectural & Methodological Innovations

Rather than submitting a single classifier notebook over an uninspected table, our project contributes five concrete engineering and research innovations spanning data governance, multi-paradigm deep & classical modeling, explainable agentic AI, and reproducible zero-server deployment:

Innovation 1: Leakage-First Contract Governance & 4-Scenario Ablation Framework

Typical student and benchmark studies feed all dataset columns directly into classifiers, inflating offline metrics via spurious artifacts. We audited `Health Insurance Fraud Claims.xlsx` (SHA-256 verified, 4,500 claims) and discovered:

- `Cluster` (`0..3`) acts as an upstream post-hoc proxy containing **267 of 270 frauds in Cluster 1** (24.27% rate vs. <= 0.26% elsewhere).
- `ClaimAmount` sits strictly at or above `$7,465.07` for all positive cases (min 7,465.07, max 9,978.43), while legitimate claims span `$100.12–$9,997.20`.

Instead of deleting evidence silently or only reporting naive 29-feature numbers, we engineered a **dual-track protocol** isolating backward compatibility (**Track A**, 29 features matching `Approch-1/` and `Approch-2-DL/` + full-stack REST API) from controlled scientific validation (**Track B**, `submission/`, running all 8 supervised models across **32 runs**: `Baseline 29f`, `Intake 28f without Cluster`, `No-Amount 27f stress test`, and `Temporal Intake >= 2024-03-11` chronological split). Removing `Cluster` lowers linear/neural PR-AUC by 6.6–10.7 percentage points (`LR` 0.9186 -> 0.8525; `SVM` 0.9252 -> 0.8181; `MLP` 0.9538 -> 0.8866), while tree ensembles (`RandomForest` AP `1.0000`, `XGBoost` AP `1.0000`) retain full ranking fidelity—even on strictly forward-in-time test claims (`>= 2024-03-11`). Crucially, dropping `ClaimAmount` proves all 8 models drop to `0.1869–0.2404` AP, quantifying exact billing-magnitude dependency with scientific honesty.

Innovation 2: Heterogeneous 10-Model Evaluation Including Negative Autoencoder Result

Inside **Track A**, we unified **5 Classical ML** (`LogisticRegression`, `RandomForest`, `XGBoost`, `SVM`, `HistGradientBoosting`) and **5 PyTorch Deep Learning** paradigms (`FraudMLP`, `ResidualMLP`, `FraudCNN1D`, `TabTransformer`, and unsupervised `Autoencoder`) behind one leakage-free scikit-learn `ColumnTransformer` (`29` features, stratified seed `42` split):

- **Attention over Categorical Context:** Our custom PyTorch `TabTransformer` embeds categorical/binary feature indicators cleanly alongside residual dense continuous layers, achieving **99.11% PR-AUC** and **99.94% ROC-AUC** (`F1 = 89.89%` at threshold `0.50`, `100.00%` test recall), leading all neural architectures in ranking quality.
- **Controlled Negative Result Rigor:** Rather than concealing poor unsupervised anomaly detection numbers, we trained a symmetric `29 -> 16 -> 8 -> 16 -> 29` `Autoencoder` solely on legitimate claims using reconstruction MSE threshold tau_AE = 0.128588 (95th validation percentile). It scored `PR-AUC = 5.23%`, `ROC-AUC = 46.02%`, and `F1 = 2.99%` (`TN=609, FP=26, FN=39, TP=1`) because frauds overlap the dense interior distribution of legitimate covariates—providing a crisp empirical lesson on supervised vs. reconstruction objectives.

Innovation 3: Dual-Path Production Serving — Three-Tier Full-Stack + Exact-Parity Browser Inference

1. **Three-Tier Service Platform (Path A):**

- `frontend/` (React 19, TypeScript, Tailwind CSS v4, Vite on `:5173`): Interactive Home Overview, **Classical ML Explorer** (5 models + offline fallback), **Deep Learning Workbench** (all 5 DL models + threshold gauge), **Agentic Pipeline Visualizer**, and **Gemini Co-Pilot Chatbox**.
- `backend/node-api/` (Node.js + Express + TypeScript on `:3000`): Unified REST API gateway proxying ML/DL inference, orchestrating `/agentic/predict`, and integrating `@google/genai` (`gemini-2.0-flash`) via `/api/chat`.
- `ml-service/` (Python FastAPI + Uvicorn on `:8000`): Dedicated stateful model server loading frozen `joblib` pipelines and PyTorch `.pt` weights (`classical_service.py`, `dl_service.py`), handling Pydantic schema validation (`ClaimFeaturesRequest`), temporal/dummy alignment, and raw vs. binary predictions.

2. **Browser-Native Pure-JS Claim Fraud Lab (Path B, `submission/`):**

- `export_browser.py` compiles fitted `ColumnTransformer` scalars/categories, logistic hyperplanes, Random Forest/XGBoost/HistGradientBoosting tree split arrays, exact RBF support-vector dual coefficients, and PyTorch `MLP`/`ResidualMLP`/`CNN1D` state-dict weights directly into compact JSON weights (`web/assets/models.json`).
- Verified end-to-end by `verify.py` and `browser_parity.json` against Python reference execution: `1,350` **preprocessing checks with** `0` **mismatches**, and `10,800` **floating-point score comparisons (**`8 models x 1,350 val+test rows`**) with maximum absolute error 4.42 x 10^-7 and** `0` **decision mismatches**, allowing zero-latency offline or static-CDN deployment (`https://claim-fraud-lab-ravi-2026.clearbell16.chatgpt.site` or `:8080`).

Innovation 4: Transparent Agentic Workflow, Cryptographic XAI Scaffold & Live Gemini Co-Pilot

Our third pillar (**Approach 3**) addresses investigator interpretability and workflow ergonomics:

- Active Node runtime (`POST /agentic/predict` in `backend/node-api/src/agentic/predict.ts`) returns a structured 4-stage audit log (`Claim Ingestion`, `Feature Validation`, `Risk Scorer`, `XAI Explainer`) under `model_used: "AgenticEnsemble-v1"`.
- Standalone microservice (`Approach-3-Agentic-AI/app/services/xai_service.py` on `:8020`) defines `XAIExplainer` providing magnitude-normalized directional feature attributions and tamper-evident **SHA-256 audit log hashes** (`log_audit`).
- React frontend (`AgenticDashboard.tsx`, `AgenticPage.tsx`, `ChatBox.tsx`) visualizes a complete 6-agent investigation pipeline (`Data Collector`, `Claim Analyzer`, `Pattern Detector`, `Billing Review`, `Provider Profiler`, `Coordinator`) and connects users to **Google Gemini 2.0 Flash** (`POST /api/chat`) for real-time explanation of flagged attributes and investigative next steps.
- *Engineering Transparency:* We explicitly distinguish the deterministic heuristic formula inside the active `/agentic/predict` handler from live FastAPI ML/DL inference (`ml-service`) and planned LangGraph + OCR + RAG extensions.

Innovation 5: Validation-Tuned Operating Thresholds + Stratified Bootstrap Uncertainty Quantification

Rather than accepting arbitrary `0.50` probability thresholds across uncalibrated classifiers, Track B chooses tau* strictly on the validation set (`validation_f1`), boosting neural and kernel test F1 cleanly (`SVM Baseline 0.8352 -> 0.8974`; `ResidualMLP Baseline 0.8608 -> 0.9286`), while auditing the single residual error (`Row 1124`) and reporting 1,000-replicate stratified bootstrap 95% confidence intervals (`Intake RandomForest F1 [0.9610, 1.0000]`, `Recall [0.9250, 1.0000]`, `AP [1.0000, 1.0000]`).

2. Detailed Implementation Architecture

2.1 Feature Pipeline & Parity Contracts

- **Canonical Input Contract (Raw API / Form Fields):**
- `ClaimAmount` (float > 0), `PatientAge` (int 0–120), `PatientIncome` (float >= 0), `ClaimDate` (`YYYY-MM-DD`, decomposed into `ClaimYear`, `ClaimMonth`, `ClaimDayOfWeek`), `PatientGender`, `ProviderSpecialty`, `ClaimStatus`, `PatientMaritalStatus`, `PatientEmploymentStatus`, `ClaimType`, `ClaimSubmissionMethod`, plus optional `Cluster` (`0..3`, included in Track A `29f` baseline and excluded in Track B `28f` intake).
- **Preprocessing Discipline:** `ColumnTransformer` scales 7 (Track A/Baseline) or 6 (Track B Intake) or 5 (No-Amount) numeric features using `StandardScaler()` fit strictly on `X_train`, and encodes 7 categorical columns via `OneHotEncoder(handle_unknown="ignore")` producing `29`, `28`, or `27` aligned columns.

2.2 Approach 1 — Classical ML Implementation (`Approch-1/`, `ml-service/`, `submission/`)

1. **Logistic Regression:** L2 penalty, `class_weight="balanced"`, `max_iter=1000`.
2. **Random Forest:** 200 estimators, `class_weight="balanced"`, `random_state=42` (`0.61s` training in Intake; selected default champion `AP = 1.0000`, `F1 = 0.9873`, `tau* = 0.4167`).
3. **XGBoost (`XGBClassifier`):** 200 trees, `max_depth=5`, `learning_rate=0.05`, `subsample=0.8`, `colsample_bytree=0.8`, `scale_pos_weight=15.67`, `eval_metric="aucpr"`.
4. **Support Vector Machine (`SVC`):** RBF kernel, `C=1.0`, `gamma="scale"`, `class_weight="balanced"`, `probability=True` (dual Platt/sigmoid scaling).
5. **Histogram Gradient Boosting (`HistGradientBoostingClassifier`):** `max_iter=200`, `learning_rate=0.08`, `early_stopping=False` (achieves `AP = 1.0000` and `F1 = 1.0000` on chronological `Temporal Intake`).

2.3 Approach 2 — Deep Learning Implementation (`Approch-2-DL/`, `ml-service/`)

All five neural models share standardized PyTorch checkpoints in `Approch-2-DL/models/` loaded dynamically by `ml-service/app/services/dl_service.py`:

1. `FraudMLP` (`dl_common.py`): `Linear(d -> 64) -> BatchNorm1d -> ReLU -> Dropout(0.2) -> Linear(64 -> 32) -> BatchNorm1d -> ReLU -> Dropout(0.2) -> Linear(32 -> 1)`. Early-stopped best epoch `34` on Track A (`PR-AUC = 95.38%`, `ROC-AUC = 99.73%`, `0.73ms` per 675 test claims).
2. `ResidualMLP` (`dl_advanced_models.py`): Input projection (`d -> 64`) followed by pre-activated residual bottleneck/identity blocks (`LayerNorm`/`BatchNorm`, ReLU, Dropout skip connections) enabling deeper gradient propagation (`PR-AUC = 95.61%` Track A, `0.67ms` batch inference).
3. `FraudCNN1D` (`submission_models.py` & `dl_advanced_models.py`): Reshapes 1D structured feature vector `(B, 1, d)` into multi-layer 1D convolution filters capturing local adjacent numeric/categorical co-occurrences + adaptive global pooling (`PR-AUC = 98.82%`, `ROC-AUC = 99.92%`, `F1 = 90.91%`, `0.77ms` batch inference).
4. `TabTransformer` (`dl_advanced_models.py`): Projects discrete/categorical columns into contextual token embeddings refined via Transformer multi-head self-attention (`TransformerEncoderLayer`), concatenated with normalized continuous projections into a classification head (`PR-AUC = 99.11%`, `ROC-AUC = 99.94%`, `18.65ms` batch inference).
5. `Autoencoder` (`dl_advanced_models.py`): Symmetric compressor (`29 -> 16 -> 8 -> 16 -> 29`) trained exclusively on `y = 0` claims with MSE loss; scored via per-sample squared reconstruction error against 95th validation percentile tau_AE = 0.128588.

2.4 Approach 3 & Full-Stack Integration (`backend/node-api/`, `frontend/`, `Approach-3-Agentic-AI/`)

- **CORS & Proxy Orchestration (`backend/node-api/src/server.ts`):** Express `:3000` exposes `/health`, `/api/ml/*` (proxying `:8000/predict/classical/*` and `:8000/predict/dl/*` with graceful error wrapping), `/agentic/predict`, and `/api/chat`.
- **Resilient Frontend UX (`frontend/src/`):** If the FastAPI `:8000` backend is stopped locally, `ClassicalMLPage.tsx` provides built-in deterministic demo fallback scoring with clear status indicators so UI workflows remain inspectable even offline, whereas running the full 3-tier launch (`ml-service` + `node-api` +

`frontend`) executes true live scikit-learn & PyTorch weights.

- **Claim Fraud Lab Static App (`submission/web/`):** Zero-dependency HTML/CSS/Vanilla-JS interface implementing exact StandardScaler + OHE transform, logistic dot products, Random Forest / XGBoost / HistGradientBoosting tree traversal, RBF kernel summation, and PyTorch forward passes (`MLP`, `ResidualMLP`, `CNN1D`) directly in the browser tab.

Supporting Visual Architecture & Quantitative Figures

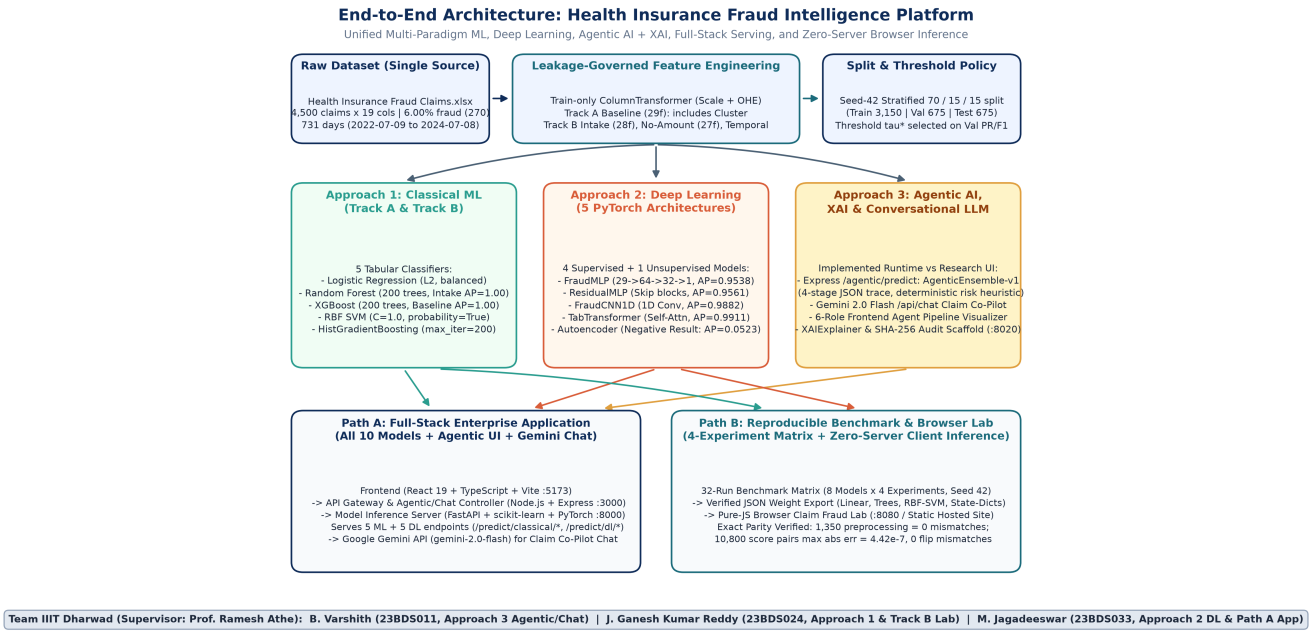


Figure 1: Dual-path architecture bridging Track A microservices & Track B zero-server browser lab.

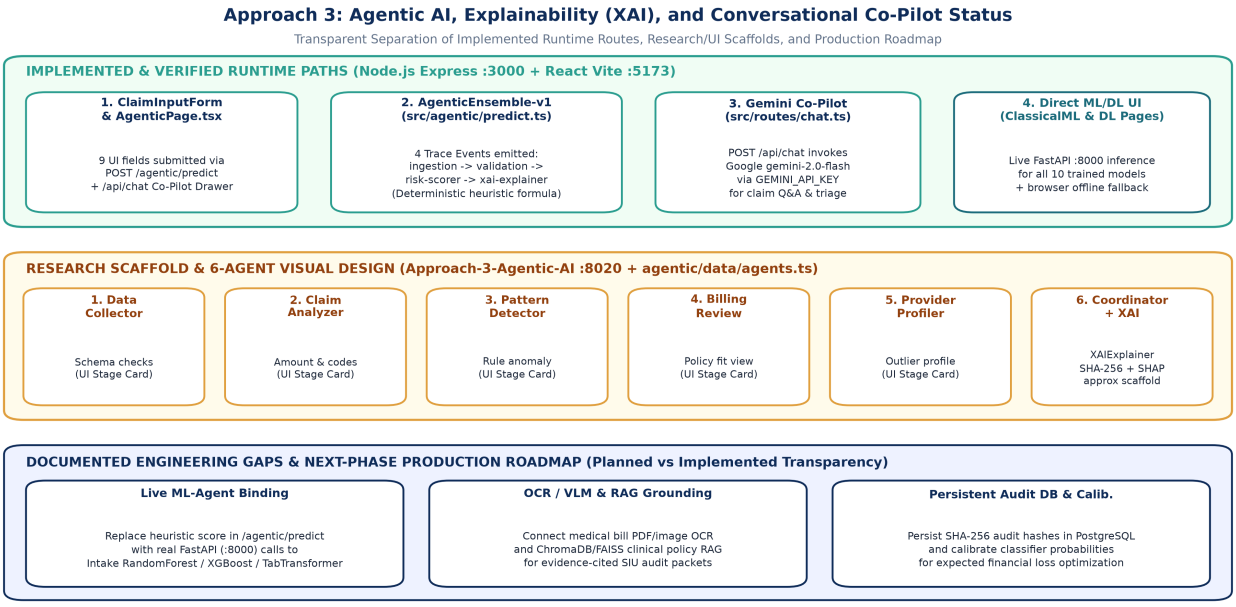


Figure 2: Approach 3 implemented runtime routes vs. 6-agent UI view & roadmap.